# A Mathematical Derivation of an Observability-Coupled Aerial–Ground Cooperative Tracking System: Event-Triggered Tube MPC and Belief-Space Guidance

Theoretical companion to the `ugv_uav_control` package

June 22, 2026

**Abstract**

This document is the standalone theoretical companion to the code report (`report.html`) for the `ugv_uav_control` ROS 2 package. It derives, from textbook foundations, the mathematics of a cooperative system in which a single downward-facing camera on a quadrotor (UAV) is the only sensor for a ground vehicle (UGV). The work extends Ferreira Santos et al. [1] in two ways: (i) the UAV trajectory is planned online by event-triggered Tube Model Predictive Control (MPC) rather than tracking fixed waypoints, and (ii) the Extended Kalman Filter (EKF) covariance is injected into both controllers' cost functions, creating a bidirectional perception–control loop. Throughout, we separate the *idealized* theory (constrained robust MPC with a minimal robust positively invariant set) from what the implementation *actually* computes (an ellipsoidal tube with loaded/fallback gains, a single-integrator UAV model, and a view-quality-scaled measurement noise). Deviations are catalogued in Section 12. We state the stability and recursive-feasibility results we rely on and are explicit about which guarantees are proven, which are inherited from cited theorems under assumptions, and which are only empirically supported.

## Contents

**1 Notation and symbol table** **2**

**2 System representations** **3**
2.1 UGV model . . . . . 3
2.2 UAV model . . . . . 4

**3 Reference and disturbance models** **5**

**4 Constraints** **5**

**5 Standard MPC formulation** **6**

**6 Tube MPC — derivation from basics** **6**
6.1 Nominal system and error dynamics . . . . . 6
6.2 Choice of $K$ . . . . . 7
6.3 Minimal robust positively invariant (mRPI) set . . . . . 7
6.4 Constraint tightening . . . . . 7
6.5 The nominal planning problem (as implemented) . . . . . 8

**7 Event-triggered Tube MPC** **8**

**8 Extended Kalman Filter with view-quality coupling** **10**
8.1 Prediction . . . 10
8.2 Process Jacobian . . . 10
8.3 Update (Joseph form) . . . 10
8.4 View-quality-coupled measurement noise . . . 11

**9 Belief-space MPC for the UGV** **11**
9.1 Covariance propagation with FoV gating . . . 11
9.2 Cost function (as implemented) . . . 11
9.3 Resulting NLP. . . . 12

**10 View-quality metric and altitude adaptation** **12**

**11 Validation: stability, recursive feasibility, and ISS** **13**

**12 Implementation notes: theory vs. code** **13**

# 1 Notation and symbol table

All symbols are defined once here and again at first use. "dim" gives the object type; $n$ denotes a scalar, $\mathbb{R}^n$ a vector, $\mathbb{R}^{n\times m}$ a matrix. Time index $k$ is discrete.

| Symbol | Meaning | Dim / units | First used |
|---|---|---|---|
| | *UGV state and model* | | |
| $x_{\text{ugv}} = [x, y, \theta, v, \omega]^\top$ | UGV state (pose + body velocities) | $\mathbb{R}^5$; m, rad, m/s, rad/s | Section 2 |
| $u_{\text{ugv}} = [v_{\text{cmd}}, \omega_{\text{cmd}}]^\top$ | UGV velocity command | $\mathbb{R}^2$; m/s, rad/s | Section 2 |
| $\alpha_v, \alpha_\omega$ | actuator-lag rates ($1/\tau$) | 2.0, 5.0 $\text{s}^{-1}$ | Section 2 |
| $\bar{v}, \bar{\omega}$ | trapezoidal mean velocities | m/s, rad/s | Section 2 |
| $F_k = \partial f/\partial x$ | UGV process Jacobian | $\mathbb{R}^{5\times5}$ | Section 8 |
| $Q$ | UGV process-noise covariance | $\mathbb{R}^{5\times5}$ | Section 8 |
| | *UAV state and model* | | |
| $p = [p_x, p_y, p_z]^\top$ | UAV position | $\mathbb{R}^3$; m | Section 2 |
| $u = [v_x, v_y, v_z]^\top$ | UAV velocity command (control) | $\mathbb{R}^3$; m/s | Section 2 |
| $A_d, B_d$ | discrete UAV dynamics matrices | $I_3$, $\Delta t\, I_3$ | Section 2 |
| $\bar{p}_k, \bar{u}_k$ | nominal UAV state / input | $\mathbb{R}^3$ | Section 6 |
| $e_k = p_k - \bar{p}_k$ | UAV tracking error | $\mathbb{R}^3$; m | Section 6 |
| $K$ | ancillary feedback gain | $\mathbb{R}^{3\times3}$ | Section 6 |
| $P_0$ | tube ellipsoid shape matrix | $\mathbb{R}^{3\times3}$, $\succ 0$ | Section 6 |
| $\alpha_k$ | adaptive tube radius | scalar | Section 6 |
| $w_k$ | additive disturbance | $\mathbb{R}^3$, $w_k \in \mathbb{W}$ | Section 3 |
| $\mathbb{W}, \Omega$ | disturbance set, (m)RPI set | compact $\subset \mathbb{R}^3$ | Section 3, Section 6 |
| $g$ | UGV predicted goal for UAV planner | $\mathbb{R}^3$; m | Section 6 |
| | *Estimation and perception* | | |
| $\mu, \Sigma$ | EKF mean, covariance | $\mathbb{R}^5$, $\mathbb{R}^{5\times5}$ | Section 8 |
| $H$ | measurement matrix | $\mathbb{R}^{m\times5}$ | Section 8 |
| $R_k, R_{\text{full}}, R_{\text{odom}}$ | measurement-noise covariances | matrices | Section 8 |
| $q_k \in (0, 1]$ | view-quality score | scalar | Section 10 |
| $d$ | horizontal UAV→UGV distance | m | Section 10 |

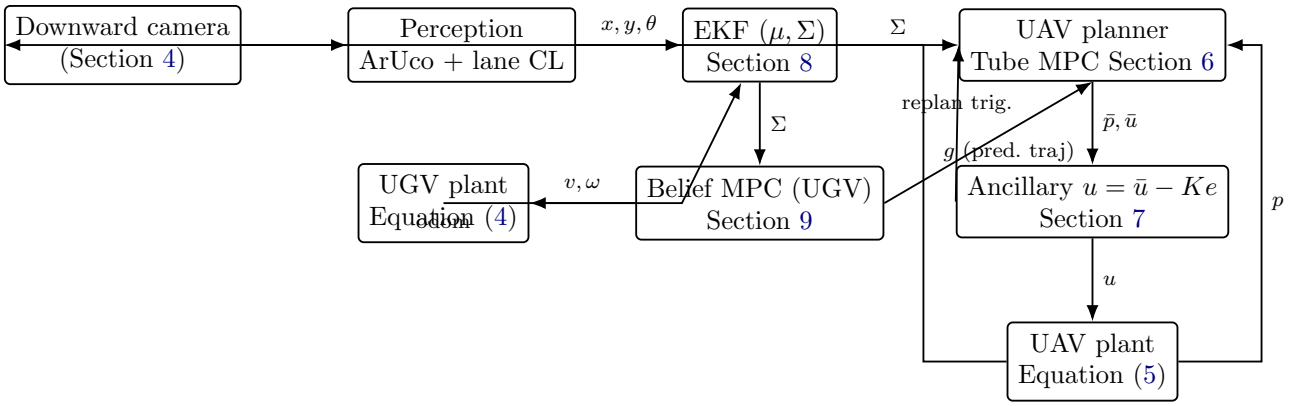



Figure 1: Controller architecture and the two coupling paths. The camera is the only sensor; perception feeds the EKF, whose covariance $\Sigma$ enters *both* the belief MPC (UGV) and the UAV Tube-MPC planner. The belief MPC's predicted trajectory is the goal $g$ for the UAV planner; the ancillary loop tracks the nominal plan and requests replans on events. Arrows back to the camera close the perception loop.

| Symbol | Meaning | Dim / units | First used |
|---|---|---|---|
| $r_{\text{fov}} = z \tan(\text{fov}/2)$ | ground FoV radius | m | Section 4 |
| fov | camera full field-of-view angle | 1.047 rad (60°) | Section 4 |
| $z_{\text{opt}}, \sigma_z$ | precision-optimal altitude, its spread | 5 m, 3 m | Section 10 |
| $\sigma_{xy} = r_{\text{fov}}/2$ | centering spread | m | Section 10 |
| $\beta$ | sigmoid sharpness (FoV roll-off) | 5 | Section 10 |
| | *Cost weights and horizons* | | |
| $N$ | prediction horizon (UAV 10, UGV 5) | steps | Section 5 |
| $\Delta t$ | time step (0.2 s MPC, 0.05 s EKF) | s | Section 5 |
| $w_\ell = \text{linspace}(1, 2, N)$ | UAV horizon weight vector | $\mathbb{R}^N$ | Section 6 |
| $w_z, w_{\text{alt}}$ | UAV altitude / visibility weights | 3, 50 | Section 6 |
| $w_d, w_h, w_r, \lambda, w_{\text{fov}}$ | UGV cost weights | $10, 10, 5, 1, 2$ | Section 9 |
| $c_{\text{fov}}(\cdot)$ | FoV-exit penalty (hinge$^2$) | scalar | Section 9 |

**Convention.** Bars $(\bar{p}, \bar{u})$ denote nominal (planned) quantities; hats are not used. $\|\cdot\|$ is the Euclidean norm; $\|e\|_{P_0}^2 := e^\top P_0 e$. $\ominus$ is the Pontryagin (Minkowski) set difference. A matrix is *Schur* if all eigenvalues lie strictly inside the unit disc.

# 2 System representations

## 2.1 UGV model

Ferreira Santos et al. [1] model the ground vehicle as a front-steered bicycle. Writing the rear-axle position $(x, y)$, heading $\theta$, longitudinal speed $v$, and steering angle $\delta$ with wheelbase $L$, the kinematic bicycle model (their Eq. 6) is

$$\dot{x} = v \cos\theta, \qquad \dot{y} = v \sin\theta, \qquad \dot{\theta} = \frac{v}{L} \tan\delta. \tag{1}$$

Under the no-slip and small-steering assumptions used by ECC (lateral forces negligible, $\tan\delta \approx$

$L\omega/v$ for commanded yaw rate $\omega$), Equation (1) reduces to the unicycle (their Eq. 7)

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega. \tag{2}$$

The TurtleBot3 is differential-drive, so Equation (2) is exact for it (no wheelbase appears).

**Discrete-time form actually implemented.** The code (`ugv_model.py`) augments Equation (2) with *first-order actuator lag*: the commanded velocities are not reached instantaneously, so $v$ and $\omega$ become states. With step $\Delta t$, lag rates $\alpha_v = 2$ ($\tau_v = 0.5\,\mathrm{s}$) and $\alpha_\omega = 5$ ($\tau_\omega = 0.2\,\mathrm{s}$),

$$v_{k+1} = v_k + \alpha_v\,(v_{\mathrm{cmd}} - v_k)\,\Delta t, \qquad \omega_{k+1} = \omega_k + \alpha_\omega\,(\omega_{\mathrm{cmd}} - \omega_k)\,\Delta t. \tag{3}$$

Pose is then integrated with the *trapezoidal mean* velocities $\bar{v} = \frac{1}{2}(v_k + v_{k+1})$, $\bar{\omega} = \frac{1}{2}(\omega_k + \omega_{k+1})$, using the exact constant-curvature arc solution

$$\boxed{\begin{aligned} x_{k+1} &= x_k + \frac{\bar{v}}{\bar{\omega}}\big(\sin(\theta_k + \bar{\omega}\,\Delta t) - \sin\theta_k\big), \\ y_{k+1} &= y_k - \frac{\bar{v}}{\bar{\omega}}\big(\cos(\theta_k + \bar{\omega}\,\Delta t) - \cos\theta_k\big), \\ \theta_{k+1} &= \theta_k + \bar{\omega}\,\Delta t, \end{aligned}} \tag{4}$$

valid for $|\bar{\omega}| \geq 10^{-4}$; otherwise the straight-line limit $x_{k+1} = x_k + \bar{v}\cos\theta_k\,\Delta t$, $y_{k+1} = y_k + \bar{v}\sin\theta_k\,\Delta t$ is used. Equation (4) follows from integrating Equation (2) over $[k\Delta t, (k{+}1)\Delta t]$ at constant $\bar{v}, \bar{\omega}$, and is exact for that segment (it avoids Euler drift). The full state is

$$x_{\mathrm{ugv}} = [x,\, y,\, \theta,\, v,\, \omega]^\top \in \mathbb{R}^5, \qquad u_{\mathrm{ugv}} = [v_{\mathrm{cmd}},\, \omega_{\mathrm{cmd}}]^\top \in \mathbb{R}^2.$$

## 2.2 UAV model

*The implementation does not use a double integrator.* Both the planner (`uav_mpc_node.py`) and the ancillary tracker (`uav_ancillary_node.py`) treat the quadrotor as a **point-mass single integrator** whose control input is the commanded *velocity* (the Gazebo `VelocityControl` plugin closes the underlying attitude/thrust loops). With position $p = [p_x, p_y, p_z]^\top$ and velocity command $u = [v_x, v_y, v_z]^\top$,

$$\boxed{p_{k+1} = p_k + \Delta t\, u_k + w_k} \qquad \Longrightarrow \qquad A_d = I_3, \quad B_d = \Delta t\, I_3, \tag{5}$$

with $w_k \in \mathbb{W}$ the disturbance (Section 3). The planner enforces velocity bounds $v_x, v_y \in [-1.5, 1.5]$ m/s and $v_z \in [-1, 1]$ m/s. Vertical velocity is a genuine decision variable, so altitude is planned, not fixed — the central modelling difference from ECC, where altitude is constant and the UAV merely tracks pre-set waypoints.

> **Remark (idealized vs. implemented).** A second-order $[p, v]^\top$ double-integrator with acceleration input $a = [a_x, a_y, a_z]^\top$ and gravity compensation, $\dot{p} = v$, $\dot{v} = a - g e_3$, is the more faithful rigid-body abstraction and is what a from-scratch Tube-MPC design would normally use. We retain the single-integrator form Equation (5) throughout because that is what the code optimizes and tracks; the double-integrator alternative is noted only here and in Section 12.

## 3 Reference and disturbance models

**Reference trajectory.** The UAV nominal trajectory $\{\bar{p}_k, \bar{u}_k\}_{k=0}^{N}$ is *not* externally given (as in ECC); it is produced online by the planner of Section 6. The planner's *goal* $g \in \mathbb{R}^3$ is the UGV's predicted position at the end of the UGV's own horizon, read from the `/ugv_predicted_traj` message published by the belief-space MPC (Section 9). Thus the reference for the aerial loop is itself the output of the ground loop's predictive model — one direction of the perception–control coupling.

**Disturbance set.** We model unmodelled velocity-tracking error, wind, and plugin latency as a bounded additive disturbance $w_k \in \mathbb{W}$. For the robust-MPC theory of Sections 6 and 11 we make the standard assumption [3]:

(A1) $\mathbb{W} \subset \mathbb{R}^3$ is compact, convex, and contains the origin: $0 \in \mathbb{W}$.

The code does not estimate $\mathbb{W}$ explicitly; the tube ellipsoid and its adaptive radius $\alpha_k$ (Section 6) act as a surrogate for the resulting invariant set. This is one of the deviations recorded in Section 12.

## 4 Constraints

**State constraints $\mathbb{X}$.** The natural workspace constraints are altitude bounds $z_{\min} \le p_z \le z_{\max}$ with $z_{\min} = 3,\ z_{\max} = 10$ m (declared in `uav_mpc_node.py`). *These are declared but never enforced as optimization constraints in the code* (Section 12); altitude is shaped only softly through the cost.

**Input constraints $\mathbb{U}$.** UAV velocity box $\mathbb{U}_{\text{uav}} = \{u : |v_x| \le 1.5, |v_y| \le 1.5, |v_z| \le 1\}$ (planner) and an ancillary saturation $u \in [-1, 1]^3$. UGV input box $\mathbb{U}_{\text{ugv}} = \{(v, \omega) : 0 \le v \le 0.22,\ |\omega| \le 2\}$.

**Field-of-view constraint.** The single perception requirement is that the UGV remain inside the downward camera's footprint. For a camera at altitude $z$ looking straight down with full opening angle fov, the imaged ground region is a disc of radius

$$r_{\text{fov}}(z) = z \tan\left(\frac{\text{fov}}{2}\right), \tag{6}$$

derived from the pinhole geometry: the extreme ray makes half-angle fov/2 with the optical (vertical) axis, so it meets the ground plane at horizontal offset $z \tan(\text{fov}/2)$. With $p^{xy} = [p_x, p_y]^\top$ the UAV ground projection and $p_{\text{ugv}} \in \mathbb{R}^2$ the UGV position, the geometric visibility condition is

$$\boxed{d := \|p^{xy} - p_{\text{ugv}}\| \ \le \ r_{\text{fov}}(z)} \tag{7}$$

i.e. the target lies within the ground footprint. The code treats Equation (7) as a *soft* constraint everywhere (a penalty in the cost, Sections 6 and 9), never as a hard constraint; Equation (6) uses the single full angle fov = 1.047, implicitly approximating the $640 \times 480$ (non-square) frame's footprint by a circle.

**Other implicit constraints.** A blocking sanity-check parameter can suppress ArUco measurements over a time window (dropout-robustness test); the perception pixel→metre scale is computed once assuming $z = 5$ m (Section 10, Section 12).

# 5 Standard MPC formulation

We recall nominal MPC [2, 7] as the object we build on. Given the discrete model $x_{k+1} = f(x_k, u_k)$, stage cost $\ell(\cdot)$, terminal cost $V_f$, and constraint sets $\mathbb{X}, \mathbb{U}$, the finite-horizon optimal control problem solved at state $x$ is

$$\begin{aligned} \min_{\mathbf{u}=\{u_0,\dots,u_{N-1}\}} \ & \sum_{k=0}^{N-1} \ell(x_k, u_k) + V_f(x_N) \\ \text{s.t.} \quad & x_0 = x, \quad x_{k+1} = f(x_k, u_k), \\ & x_k \in \mathbb{X},\ u_k \in \mathbb{U},\ x_N \in \mathbb{X}_f. \end{aligned} \tag{8}$$

In receding-horizon implementation one applies only the first input $u_0^\star(x)$, advances the plant, and re-solves at the next state. With a control-invariant terminal set $\mathbb{X}_f$ and terminal cost that is a local Lyapunov function, Equation (8) is recursively feasible and the origin is asymptotically stable [7, Thm. 2.19]. The remaining sections specialise this template: Section 6 makes it *robust* to $w_k$, and Section 9 augments the stage cost with estimation uncertainty.

# 6 Tube MPC — derivation from basics

Tube MPC [3] separates the controller into (i) a *nominal* plan computed by MPC on the disturbance-free system, and (ii) an *ancillary* feedback law that keeps the true state inside a "tube" around the nominal plan despite $w_k$. We derive both for the single-integrator UAV Equation (5).

## 6.1 Nominal system and error dynamics

The nominal (disturbance-free) system is

$$\bar{p}_{k+1} = A_d\, \bar{p}_k + B_d\, \bar{u}_k = \bar{p}_k + \Delta t\, \bar{u}_k. \tag{9}$$

Define the tracking error $e_k := p_k - \bar{p}_k$. The ancillary law adds proportional feedback on the error:

$$\boxed{u_k = \bar{u}_k - K\, e_k} \tag{10}$$

(`uav_ancillary_node.py`; the command is then saturated to $[-1, 1]^3$). Subtracting Equation (9) from the true dynamics Equation (5) and substituting Equation (10),

$$\begin{aligned} e_{k+1} &= (p_k + \Delta t\, u_k + w_k) - (\bar{p}_k + \Delta t\, \bar{u}_k) \\ &= e_k + \Delta t\, (u_k - \bar{u}_k) + w_k = e_k - \Delta t\, K e_k + w_k \\ &= \underbrace{(I_3 - \Delta t\, K)}_{=:A_K}\, e_k + w_k. \end{aligned} \tag{11}$$

Robust regulation of the error requires $A_K = A_d - B_d K = I_3 - \Delta t K$ to be *Schur*. With the diagonal gain actually used (fallback $K = 2I_3$, $\Delta t = 0.2$), $A_K = 0.6\, I_3$, whose eigenvalues 0.6 lie inside the unit disc, so Equation (11) is exponentially stable in the nominal ($w = 0$) case. Any loaded gain $K$ must satisfy the same Schur condition.

### 6.2 Choice of $K$

In the textbook design $K$ is the discrete-time LQR gain minimising $\sum_k (e_k^\top Q_e e_k + u_k^\top R_e u_k)$, i.e. $K = (R_e + B_d^\top P B_d)^{-1} B_d^\top P A_d$ with $P$ the solution of the discrete algebraic Riccati equation. *This computation is not performed in the repository.* The gain $K$ and the tube shape matrix $P_0$ are loaded from a precomputed file (`tube_params.npz`); when absent, the node falls back to $K = 2I_3$ and $P_0 = 6.86\, I_3$. We therefore treat $K, P_0$ as given design data satisfying the Schur and positive-definiteness requirements, and record in Section 12 that their principled (LQR/RPI) derivation belongs to an upstream stage absent from these sources.

### 6.3 Minimal robust positively invariant (mRPI) set

For the error system Equation (11) with $A_K$ Schur and $w_k \in \mathbb{W}$ (A1), a set $\Omega \subset \mathbb{R}^3$ is *robust positively invariant (RPI)* if

$$A_K\, \Omega \oplus \mathbb{W} \subseteq \Omega, \tag{12}$$

where $\oplus$ is the Minkowski sum. The *minimal* RPI set is the limit

$$\Omega_\infty = \bigoplus_{i=0}^{\infty} A_K^i\, \mathbb{W}, \tag{13}$$

which converges because $A_K$ is Schur (the tail $\sum_{i \geq n} \left\| A_K^i \right\|$ is geometrically summable) [6]. $\Omega_\infty$ is the smallest set such that $e_0 \in \Omega_\infty \Rightarrow e_k \in \Omega_\infty\ \forall k$ for every disturbance sequence; it is compact and contains the origin. In practice one computes an outer $\varepsilon$-approximation $\Omega \supseteq \Omega_\infty$ by truncating Equation (13) [6].

**Ellipsoidal surrogate actually used.** Rather than computing Equation (13), the code represents the tube by the *ellipsoid*

$$\boxed{\mathcal{E}(\alpha_k) = \{\, e \in \mathbb{R}^3 : e^\top P_0\, e \leq \alpha_k^2 \,\}} \tag{14}$$

and declares the state "inside the tube" iff $e_k \in \mathcal{E}(\alpha_k)$. If $P_0$ is chosen as a Lyapunov matrix for $A_K$ (i.e. $A_K^\top P_0 A_K - P_0 \prec 0$) then a sublevel set $\{e^\top P_0 e \leq c\}$ is positively invariant for the nominal error dynamics, and for a suitable $c = \alpha^2$ accounting for $\mathbb{W}$ it is RPI; Equation (14) is the implementation's stand-in for $\Omega$. Crucially the radius *adapts to estimation quality* (the extension over ECC):

$$\boxed{\alpha_k = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min})\ \mathrm{clip}\left( \frac{\mathrm{tr}\, \Sigma_{xy} - \mathrm{tr}_{\min}}{\mathrm{tr}_{\max} - \mathrm{tr}_{\min}},\ 0,\ 1 \right)} \tag{15}$$

with $\alpha_{\min} = 0.5$, $\alpha_{\max} = 2$, $\mathrm{tr}_{\min} = 0.005$, $\mathrm{tr}_{\max} = 0.15$, and $\mathrm{tr}\, \Sigma_{xy} = \Sigma_{xx} + \Sigma_{yy}$ the EKF horizontal position-covariance trace (Section 8). The tube *breathes*: it tightens when the position estimate is sharp and loosens when perception degrades.

### 6.4 Constraint tightening

The nominal MPC must leave room for the error to occupy $\Omega$. Tube MPC tightens the original constraints by the Pontryagin difference [3]:

$$\bar{\mathbb{X}} = \mathbb{X} \ominus \Omega, \qquad \bar{\mathbb{U}} = \mathbb{U} \ominus K\Omega, \tag{16}$$

where $\mathbb{X} \ominus \Omega = \{x : x \oplus \Omega \subseteq \mathbb{X}\}$. Planning the nominal trajectory inside $\bar{\mathbb{X}}, \bar{\mathbb{U}}$ guarantees the true state/input stay in $\mathbb{X}, \mathbb{U}$ for any admissible disturbance, since $x_k = \bar{p}_k + e_k \in \bar{\mathbb{X}} \oplus \Omega \subseteq \mathbb{X}$. *The code does not implement Equation* (16) (it has neither explicit $\mathbb{X}, \mathbb{U}$ nor $\Omega$); we state it because it is the formal justification for why a correctly-sized tube yields constraint satisfaction, and its absence is a gap (Section 12).

### 6.5 The nominal planning problem (as implemented)

The planner optimises the velocity sequence $\mathbf{u} = \{\bar{u}_0, \ldots, \bar{u}_{N-1}\} \in \mathbb{R}^{N\times 3}$, $N = 10$, $\Delta t = 0.2$, over the single integrator. Predicted positions are

$$\bar{p}_k = \bar{p}_0 + \Delta t \sum_{j=0}^{k} \bar{u}_j, \qquad k = 0, \ldots, N-1, \tag{17}$$

(the cumulative sum of `uav_mpc_node.py`; $\bar{p}_k$ here is the position *after* $k{+}1$ steps, distinct from the base point $\bar{p}_0$). With horizon weights $w_\ell = \text{linspace}(1, 2, N)$ (later steps weighted more), $z_{\text{opt}}$ the precision-optimal altitude, $w_z = 3$, $w_{\text{alt}} = 50$, and goal $g$ (with $g^{xy}$ its horizontal part), the cost is

$$\boxed{\begin{aligned} J_{\text{uav}} = &\sum_k w_{\ell,k} \, \|\bar{p}_k - g\|^2 \; + \; w_z \sum_k w_{\ell,k} \, (z_k - z_{\text{opt}})^2 \\ &+ \; w_{\text{alt}} \sum_k w_{\ell,k} \, \frac{\left\|\bar{p}_k^{xy} - g^{xy}\right\|^2}{z_k^2 + 0.1} \; + \; 0.1 \sum_k \|\bar{u}_k\|^2 . \end{aligned}} \tag{18}$$

**Term 1** tracks the predicted UGV goal; note it is a *full* 3-*vector* norm and $g$ carries $z \approx 0$ (ground), so it also pulls altitude down and is partly opposed by Term 2. **Term 2** is the altitude regulariser toward $z_{\text{opt}}$ ($w_z$ is a plain quadratic weight — the code's `sigma_z` parameter — *not* a standard deviation). **Term 3** is the visibility heuristic: horizontal offset divided by altitude-squared, so a lateral excursion is tolerated only if the drone is high enough to keep the target in frame (cf. Equation (6)). **Term 4** penalises control effort. The problem

$$\min_{\mathbf{u}} \; J_{\text{uav}} \quad \text{s.t.} \quad \textit{Equation (17)}, \; v_x, v_y \in [-1.5, 1.5], \; v_z \in [-1, 1]$$

is solved by SLSQP [4] with `maxiter`=15, warm-started by rolling the previous solution. It is re-solved only when an event fires (Section 7); a legacy timer mode adds a 1 Hz solve and freezes $v_z = 0$.

## 7 Event-triggered Tube MPC

Standard Tube MPC re-solves the nominal problem every step. To respect a 150 ms control budget, the ancillary node runs at 20 Hz but requests a *replan* only when one of four prioritised conditions fires (`uav_ancillary_node.py`). Let $t$ be the current time and let $\tau_\ell$ be the last fire time of level $\ell$.

**Level 1a — Tube exit (critical, no lockout).** The set-membership test on the ellipsoid Equation (14):

$$\text{fire} \quad \Longleftrightarrow \quad e_k^\top P_0 \, e_k \; > \; \alpha_k^2. \tag{19}$$

This is the literal robustness trigger: the true state has left the tube, so the current plan is no longer guaranteed to contain it. It fires immediately, with no lockout.

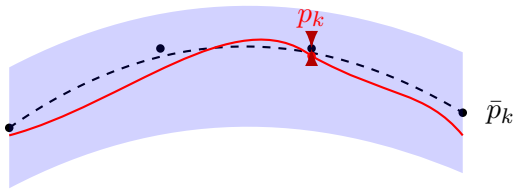



Figure 2: Tube MPC concept. The nominal plan $\bar{p}_k$ (dashed, single-integrator Equation (17)) is surrounded by the ellipsoidal tube $\mathcal{E}(\alpha_k)$ Equation (14). The ancillary law Equation (10) regulates the true state $p_k$ (red) toward $\bar{p}_k$; the tube radius $\alpha_k$ adapts to the EKF covariance Equation (15).

**Level 1b — ArUco covariance spike (1 s lockout).** A sudden estimation-quality collapse:

$$\text{fire} \iff \operatorname{tr}\Sigma_{xy}(t) > 0.12 \;\wedge\; \operatorname{tr}\Sigma_{xy}(t) > 1.5 \cdot \operatorname{tr}\Sigma_{xy}(t-\delta), \quad \delta \approx 0.5 \text{ s}, \tag{20}$$

and $t - \tau_{1b} > 1\,\text{s}$.

**Level 2 — Curvature anticipation (1 s lockout).** Over the first two segments of the UGV predicted path, estimate the discrete curvature

$$\kappa_i = \frac{|\Delta\theta_i|}{\Delta s_i + 10^{-6}}, \qquad \Delta\theta_i = \operatorname{wrap}(\theta_{i+1} - \theta_i), \quad \Delta s_i = \|p_{i+1} - p_i\|, \tag{21}$$

and fire iff $\max_i \kappa_i > 0.4$ (and $t - \tau_2 > 1\,\text{s}$). This anticipates a turn so the UAV can climb to widen the footprint before the UGV reaches it.

**Level 3 — Drift maintenance (1 s lockout).** A slow-drift safety net:

$$\text{fire} \iff \operatorname{tr}\Sigma_{xy}(t) > 0.07 \;\wedge\; (t - t_{\text{replan}}) > T_{\min}, \quad T_{\min} = 3 \text{ s}. \tag{22}$$

**Chattering avoidance and feasibility.** Levels 1b/2/3 each carry a 1 s lockout, the implementation's chattering guard (a time-dwell rather than a consecutive-violation counter). A *bootstrap* trigger requests the first plan, and a TAKEOFF→TRACKING state machine first climbs to 5 m at 0.5 m/s.

**Why event-triggering preserves robust invariance.** Between triggers the ancillary law Equation (10) is always active, so the error obeys Equation (11) with $A_K$ Schur; hence $e_k \in \mathcal{E}(\alpha_k)$ is positively invariant for as long as the plan and radius are held fixed. The critical trigger Equation (19) fires *exactly* when invariance would be lost, forcing a fresh plan recentred on the current state; under the standard recursive-feasibility argument (Section 11) this restores a feasible nominal trajectory. Triggers 1b/2/3 are *anticipatory* (they replan before the tube is violated), so they can only improve feasibility margins; their lockouts prevent solve storms without affecting the critical guarantee, which has no lockout. The argument is the standard one for event-triggered Tube MPC and relies on the tube being a true RPI set — an assumption inherited from the (absent) RPI design, see Section 12.

# 8 Extended Kalman Filter with view-quality coupling

The EKF (`ekf_node.py`) estimates the UGV belief $(\mu, \Sigma)$ over the 5-state $x_{\text{ugv}} = [x, y, \theta, v, \omega]^\top$ at a 20 Hz prediction rate ($\Delta t = 0.05\,\text{s}$), correcting asynchronously from two sources: wheel odometry and the ArUco pose recovered from the drone image.

## 8.1 Prediction

With process model $f$ Equations (3) and (4) and the last command,

$$\mu^- = f(\mu,\, u_{\text{ugv}}; \Delta t), \qquad \Sigma^- = F\,\Sigma\,F^\top + Q, \tag{23}$$

with $Q = \text{diag}(10^{-4}, 10^{-4}, 4{\times}10^{-4}, 10^{-4}, 2.5{\times}10^{-3})$.

## 8.2 Process Jacobian

The code uses the *first-order (Euler) Jacobian*

$$F = I_5 + \begin{bmatrix} 0 & 0 & -v\sin\theta\,\Delta t & \cos\theta\,\Delta t & 0 \\ 0 & 0 & v\cos\theta\,\Delta t & \sin\theta\,\Delta t & 0 \\ 0 & 0 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & -\alpha_v\,\Delta t & 0 \\ 0 & 0 & 0 & 0 & -\alpha_\omega\,\Delta t \end{bmatrix}. \tag{24}$$

The bottom two rows are the exact linearisation of the actuator lag Equation (3) ($\partial v_{k+1}/\partial v_k = 1 - \alpha_v \Delta t$, etc.). The top rows are the Jacobian of the *Euler* pose update $x_{k+1} = x_k + v\cos\theta\,\Delta t$, *not* of the exact arc model Equation (4) actually used to propagate the mean: Equation (24) drops $\partial x/\partial\omega$ and the trapezoidal-mean/lag factors in the $v$ and $\theta$ partials. For small $\Delta t$ the two Jacobians agree to first order; the mismatch is a deliberate approximation, recorded in Section 12.

## 8.3 Update (Joseph form)

For a measurement $z = h(x) + \nu$, $\nu \sim \mathcal{N}(0, R)$, with $H = \partial h/\partial x$,

$$S = H\Sigma^- H^\top + R, \qquad K_{\text{KF}} = \Sigma^- H^\top S^{-1}, \qquad \mu^+ = \mu^- + K_{\text{KF}}\,(z - H\mu^-), \tag{25}$$

(with heading innovation wrapped to $(-\pi, \pi]$), and the Joseph-form covariance update for numerical stability,

$$\boxed{\Sigma^+ = (I - K_{\text{KF}}H)\,\Sigma^-\,(I - K_{\text{KF}}H)^\top + K_{\text{KF}}\,R\,K_{\text{KF}}^\top} \tag{26}$$

followed by symmetrisation $\Sigma^+ \leftarrow \frac{1}{2}(\Sigma^+ + \Sigma^{+\top})$. Equation (26) keeps $\Sigma^+ \succeq 0$ even under marginal numerics [8].

**Measurement models.** Both are linear, so $H$ is constant.

- **Odometry** measures the body velocities, $z = [v, \omega]^\top$, with

$$H_{\text{odom}} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad R_{\text{odom}} = \text{diag}(4{\times}10^{-4},\, 2.5{\times}10^{-3}).$$

- **ArUco** provides only pose $(x, y, \theta)$. The code forms a *full-state pseudo-measurement* by padding with the current velocity estimate, $z = [x, y, \theta, \mu_v, \mu_\omega]^\top$, and uses $H = I_5$. The last two rows carry no new information (their innovation is zero by construction) and contribute only through $R$ — a modelling artefact noted in Section 12, not a true 5-dimensional observation.

### 8.4 View-quality-coupled measurement noise

*Intuition.* When the drone is far from optimal — target near the edge of the footprint or altitude far from $z_{\text{opt}}$ — the ArUco pose is unreliable and should be trusted less. The code therefore scales the ArUco noise *inversely* with the geometric view-quality score $q_k \in (0, 1]$ (Section 10):

$$\boxed{R_k = \frac{R_{\text{full}}}{q_k + 10^{-3}}} \tag{27}$$

with $R_{\text{full}} = \text{diag}(2.5{\times}10^{-3}, 2.5{\times}10^{-3}, 10^{-2}, 4{\times}10^{-4}, 2.5{\times}10^{-3})$. As $q_k \to 1$ (excellent view) $R_k \to R_{\text{full}}$; as $q_k \to 0$ (poor view) $R_k \to \infty$ and the update Equation (25) ignores the ArUco measurement. This is the second arm of the perception–control coupling: the UAV's geometry, via $q_k$, modulates how much the ground estimate trusts its own perception.

> **Note on form.** An alternative pixel-based model $R_k = R_{\text{base}}(1 + \alpha \, \|d_{\text{pixel}}\|^2)$ (noise growing with the marker's pixel distance from frame centre) is a common heuristic, but it is *not* what the code implements. The implementation uses the inverse-view-quality form Equation (27), where $q_k$ is a function of the 3-D UAV/UGV geometry, not pixel coordinates.

## 9 Belief-space MPC for the UGV

The ground controller (`belief_mpc_node.py`) is an information-aware NMPC: it tracks the lane while actively keeping itself observable. Following belief-space planning [5, 9] the planning state is augmented with the covariance,

$$\xi_k = (\mu_k, \Sigma_k), \tag{28}$$

and the controller propagates *both* the mean (via $f$, Equation (4)) and the covariance through the horizon, so that it can reason about how its own motion will change the quality of the drone's estimate.

### 9.1 Covariance propagation with FoV gating

At each predicted step the controller forms the no-observation prediction $\Sigma_{\text{pred}} = F\Sigma F^\top + Q$ and a post-measurement covariance $\Sigma_{\text{obs}}$ from a Joseph update Equation (26) with the same dynamic noise Equation (27). Whether a measurement is available depends *smoothly* on visibility through the sigmoid gate

$$\gamma = \frac{1}{1 + \exp\big(\beta \, (d - r_{\text{fov}})\big)}, \qquad \beta = 5, \tag{29}$$

($\gamma \to 1$ inside the footprint $d < r_{\text{fov}}$, $\gamma \to 0$ outside), and the predicted covariance is the blend

$$\boxed{\Sigma_k = (1 - \gamma) \, \Sigma_{\text{pred}} + \gamma \, \Sigma_{\text{obs}}} \tag{30}$$

This is the predictive heart of the method: the MPC simulates the future estimator and optimises against it.

### 9.2 Cost function (as implemented)

With horizon $N = 5$, $\Delta t = 0.2$, mean $\mu_k$, target $x^{\text{ref}} = p_{\text{tgt}}$, heading error $e_{\theta,k}$, control increment $\Delta u_k = u_k - u_{k-1}$, and weights $w_d = 10$, $w_h = 10$, $w_r = 5$, $\lambda = 1$, $w_{\text{fov}} = 2$,

$$\boxed{J_{\text{ugv}} = \sum_{k=0}^{N-1} \Big[ \, w_d \, \|p_k - p_{\text{tgt}}\|^2 + w_h \, e_{\theta,k}^2 + w_r \, \|\Delta u_k\|^2 + \lambda \, \text{tr}\, \Sigma_k + w_{\text{fov}} \, c_{\text{fov}}(x_k) \Big]} \tag{31}$$

The FoV-exit penalty is a **one-sided quadratic hinge** (a ReLU$^2$ soft constraint — not a log- or inverse-barrier):

$$c_{\text{fov}}(x_k) = \big(\max(0,\, d_k - r_{\text{fov}})\big)^2. \tag{32}$$

**Per-term rationale.** (i) $w_d$: track the lane-centreline waypoint. (ii) $w_h$: align heading toward the target — switched off near the target (when squared distance $< 0.05\,\text{m}^2$, i.e. within $\approx 0.22\,\text{m}$) to avoid spin-in-place. (iii) $w_r$: penalise control increments for smoothness. (iv) $\lambda \operatorname{tr} \Sigma_k$: the information term — prefer manoeuvres the drone can estimate well (lowers predicted uncertainty). (v) $w_{\text{fov}} c_{\text{fov}}$: directly discourage leaving the footprint Equation (7). Terms (iv)–(v) are what make the controller "belief-coupled" and are absent from ECC and from the baseline controllers (`mpc_node`: only (i)–(iii) with $w_h = 2$).

### 9.3 Resulting NLP.

The problem

$$\min_{\{u_0,\ldots,u_{N-1}\}} J_{\text{ugv}} \quad \text{s.t.} \quad \text{Equations (4) and (30)}, \quad 0 \le v \le 0.22,\ |\omega| \le 2,$$

is a nonlinear program (the covariance rollout makes it non-convex) solved by SLSQP [4] with `maxiter`=20; the first control is applied and the full rollout is published as the reference goal $g$ for the UAV planner of Section 6.

## 10 View-quality metric and altitude adaptation

The scalar that couples geometry to both the EKF (Equation (27)) and the belief MPC gate (Equation (29)) is the differentiable view-quality score $q$ (`view_quality.py`). It is the *product of three* factors:

$$q(d,z) = \underbrace{\frac{1}{1+\exp\big(\beta(d - r_{\text{fov}})\big)}}_{\text{(a) FoV-boundary gate}} \cdot \underbrace{\exp\Big(-\frac{d^2}{2\sigma_{xy}^2}\Big)}_{\text{(b) centering}} \cdot \underbrace{\exp\Big(-\frac{(z - z_{\text{opt}})^2}{2\sigma_z^2}\Big)}_{\text{(c) altitude precision}} \tag{33}$$

with $r_{\text{fov}} = z \tan(\text{fov}/2)$ Equation (6) (clamped $\ge 0.1$), $d$ the horizontal UAV→UGV distance, $\sigma_{xy} = r_{\text{fov}}/2$, $\beta = 5$, $z_{\text{opt}} = 5\,\text{m}$, $\sigma_z = 3\,\text{m}$.

**Factor (a)** is a smooth gate going $1 \to 0$ as the target crosses the footprint radius $r_{\text{fov}}$ (at $d = r_{\text{fov}}$, value $\frac{1}{2}$); $\beta$ sets the roll-off sharpness. **Factor (b)** rewards keeping the target near the optical axis, with spread tied to the footprint ($\sigma_{xy} = r_{\text{fov}}/2$). **Factor (c)** is a Gaussian preferring the precision-optimal altitude $z_{\text{opt}}$: too low loses footprint, too high loses pixel resolution. Here $\sigma_z$ *is* a genuine standard deviation (contrast the planner weight $w_z$ of Equation (18), which shares the code name `sigma_z` but is a plain quadratic weight). The product is differentiable, hence safe inside the gradient-based SLSQP solvers; a central-difference gradient helper is also provided.

**Altitude adaptation.** There is no separate altitude controller. Altitude emerges from the planner cost Equation (18): factor (c)'s preference enters as the $w_z(z - z_{\text{opt}})^2$ term, while the visibility term $w_{\text{alt}} \|\bar{p}^{xy} - g^{xy}\|^2 / (z^2 + 0.1)$ rewards climbing when the target is laterally offset (a higher $z$ enlarges $r_{\text{fov}}$ and shrinks the penalty). The curvature trigger Equation (21) causes a replan before a turn, and the resolved plan then chooses to climb if doing so lowers Equation (18). Thus "climb into turns to widen the field of view" is an *emergent* optimum, not a hand-coded rule.

**Perception scale caveat.** The pixel→metre conversion used to recover the ArUco world position, $s = 2h\tan(\text{fov}/2)/W$ (image width $W$), is computed *once* with a hard-coded $h = 5\,\text{m}$. Because altitude is in fact a decision variable, the metric scale — and the $x, y$ fed to the EKF — are correct only at $z = 5$ and are off by a factor $z/5$ otherwise (Section 12).

## 11 Validation: stability, recursive feasibility, and ISS

We are explicit about what is proven, what is inherited from cited theorems under assumptions, and what is only empirical.

**Recursive feasibility of Tube MPC.** Under (A1) and assuming $\Omega$ is a true RPI set for Equation (11) with tightened constraints Equation (16), the nominal problem is recursively feasible: a feasible plan at step $k$ yields a feasible candidate at $k{+}1$ by shifting and appending a terminal step, and the true state remains in $\bar{\mathbb{X}} \oplus \Omega \subseteq \mathbb{X}$ [3, Thm. 1][6]. *Caveat:* this guarantee requires the tube to be a correctly computed RPI set. The implementation uses the loaded/fallback ellipsoid Equation (14) with gains not derived here, and does not implement constraint tightening; so recursive feasibility holds *for the idealized design* and is assumed, not verified, for the code.

**Input-to-state stability (ISS).** For Equation (11) with $A_K$ Schur and $w_k \in \mathbb{W}$ bounded, the error is ISS: there exist a $\mathcal{KL}$ function $\beta_{\text{ISS}}$ and a $\mathcal{K}$ function $\gamma_{\text{ISS}}$ with $\|e_k\| \le \beta_{\text{ISS}}(\|e_0\|, k) + \gamma_{\text{ISS}}(\sup_i \|w_i\|)$, and the true state converges to the tube $\Omega$ around the nominal trajectory [3]. For the implemented $A_K = 0.6\, I_3$ this is immediate (a contraction with disturbance), giving $\|e_{k+1}\| \le 0.6\,\|e_k\| + \sup\|w\|$, hence $\limsup_k \|e_k\| \le \sup\|w\|\,/0.4$. The nominal MPC's own stability is the standard result of Section 5 [2, 7].

**Belief-MPC stability.** Adding $\lambda \operatorname{tr}\Sigma_k$ and $c_{\text{fov}}$ to the stage cost Equation (31) does *not* automatically preserve nominal-MPC stability guarantees: the covariance rollout Equation (30) makes the cost non-convex and state-dependent in a way the standard terminal-cost machinery does not cover. We rely on Equation (31) being a valid Lyapunov candidate under mild assumptions (bounded covariance, persistent observability) in the spirit of belief-space planning [5, 9]; **we have not proven this formally**, and the controller runs with no terminal set or terminal cost.

**Numerical validation.** The package's README reports a comparison table (mean tracking error $1.037 \to 0.463\,\text{m}$, $-55.3\%$; ArUco visibility $46.3\% \to 87.0\%$) attributing the gains to the belief coupling and adaptive tube. These figures are *empirical and external*: the code report (`report.html`) notes they originate from upstream "Phase 1/Phase 3" artifacts not contained in this package and could not be reproduced from these sources alone. We therefore cite them as indicative empirical evidence, **not** as formal proof, and recommend re-running the logger (`experiment_logger.py`) to regenerate them before relying on the numbers.

## 12 Implementation notes: theory vs. code

This section lists, honestly, where the implementation deviates from the idealized derivations above.

1. **UAV model.** Theory of robust MPC is usually posed on a double integrator with acceleration input; the code uses the *single integrator* Equation (5) with velocity input ($A_d = I_3$, $B_d = \Delta t\, I_3$). All derivations here use the implemented form.

2. **Tube gains $K, P_0$.** Not computed (no LQR/Riccati, no mRPI iteration Equation (13)). Loaded from `tube_params.npz`; absent that file, fallback $K = 2I_3$, $P_0 = 6.86\, I_3$. The Schur and Lyapunov properties assumed in Sections 6 and 11 hold for the fallback but the designed RPI set is external ("Phase 1").

3. **Constraint tightening** Equation (16) is not implemented; there are no explicit $\mathbb{X}, \mathbb{U}, \Omega$ in the code. The tube is the ellipsoid Equation (14) with adaptive radius Equation (15).

4. **Nominal/ancillary timestep mismatch.** The planner emits the plan at $\Delta t = 0.2\,\text{s}$, but the ancillary interpolator `get_nominal_state` assumes 0.3 s steps. The interpolated $\bar{p}$ and the finite-difference $\bar{u}$ are therefore time-distorted — a likely bug; the tracker follows a stretched version of the plan.

5. **EKF Jacobian.** Equation (24) is the Euler Jacobian, not the exact derivative of the arc model Equation (4) used for the mean. First-order accurate in $\Delta t$.

6. **ArUco pseudo-measurement.** $H = I_5$ with $z$'s velocity entries set to the current estimate; the $v, \omega$ rows carry no information (Section 8).

7. **Altitude limits.** $z_{\min} = 3, z_{\max} = 10$ are declared but never enforced as constraints; only the velocity bound $v_z \in [-1, 1]$ and the soft $z_{\text{opt}}$ cost shape altitude.

8. **Inconsistent $z_{\text{opt}}$.** Planner default 5.0, launch file passes 6.0, `view_quality.py` hard-codes 5.0; the planner and the metric driving the EKF can disagree on the "optimal" altitude.

9. **Perception scale.** Pixel→metre scale fixes $h = 5\,\text{m}$ once (Section 10); incorrect by $z/5$ whenever the planned altitude $z \neq 5$.

10. **Solver settings.** SLSQP with `ftol`$= 10^{-3}$; `maxiter`$= 15$ (UAV planner), 20 (UGV MPCs). Horizons $N = 10$ (UAV, $\Delta t = 0.2$), $N = 5$ (UGV, $\Delta t = 0.2$); EKF predict $\Delta t = 0.05$ (20 Hz); ancillary loop 20 Hz.

11. **Trigger form.** The tube-exit trigger is the ellipsoidal test Equation (19), not a polytopic $\max(\Omega_H e - \Omega_h) > 0$; chattering is handled by 1 s time-lockouts on levels 1b/2/3, not by an $n_{\text{thresh}}$ consecutive-violation counter.

# References


[1] Davi Ferreira Santos, Pedro Castillo, and Alessandro Corrêa Victorino. Aerial vision based guidance and control for a perception-less ground vehicle. In *Proceedings of the European Control Conference (ECC)*, 2024.

[2] David Q. Mayne, James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

[3] David Q. Mayne, María M. Seron, and Saša V. Raković. Robust model predictive control of constrained linear systems with bounded disturbances. *Automatica*, 41(2):219–224, 2005.

[4] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.

[5] Robert Platt, Russ Tedrake, Leslie Kaelbling, and Tomás Lozano-Pérez. Belief space planning assuming maximum likelihood observations. In *Proceedings of Robotics: Science and Systems (RSS)*, 2010.

[6] Saša V. Raković, Eric C. Kerrigan, Konstantinos I. Kouramas, and David Q. Mayne. Invariant approximations of the minimal robust positively invariant set. *IEEE Transactions on Automatic Control*, 50(3):406–410, 2005.

[7] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2nd edition, 2017.

[8] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.

[9] Jur van den Berg, Sachin Patil, and Ron Alterovitz. Motion planning under uncertainty using iterative local optimization in belief space. *International Journal of Robotics Research*, 31(11): 1263–1278, 2012.